

Raftar Application

Autonomous

Submitted by:-

Divyanshu D Gedam
EE25B046

Main Repository- https://github.com/DiscoJK414/Raftaar_App

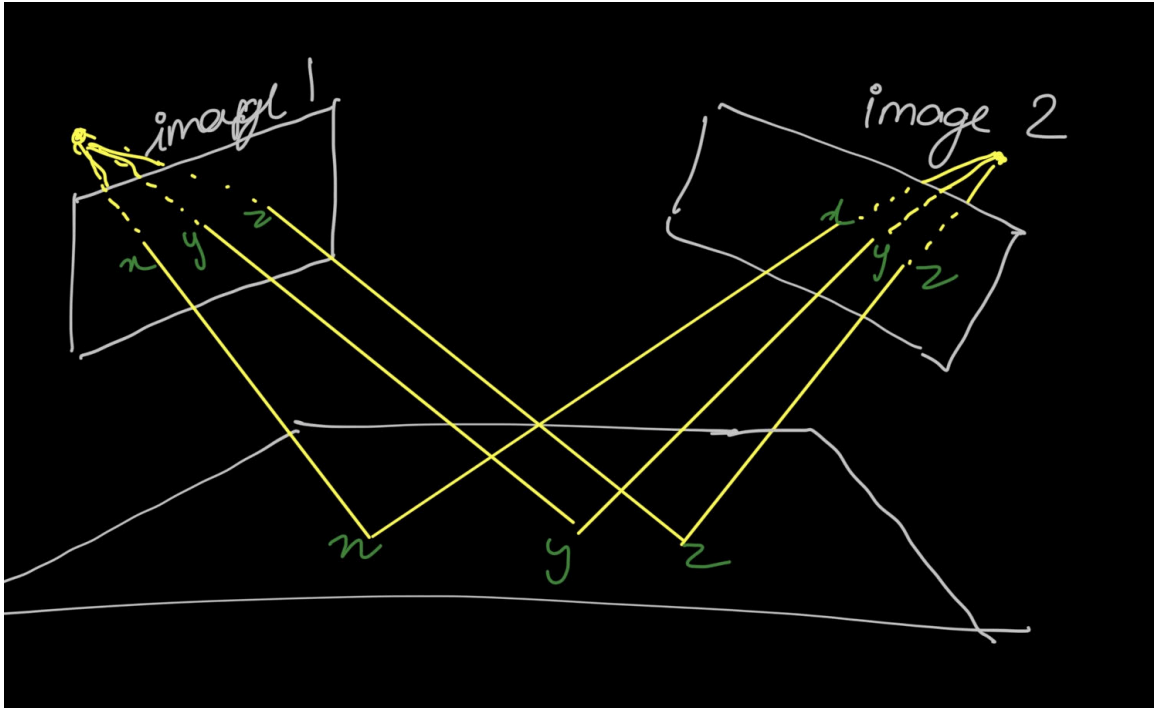
Q1)SKIBIDI STEREO

CODE- https://github.com/DiscoJK414/Raftaar_App/tree/main/Q1

a) Stereo fusion

Homography is a concept in computer vision which helps us align the planar perspectives of the same surface from different perspectives. It helps us to map features and points from different images (from different viewpoints).

A Homography matrix is a transformation matrix, which relates the pixel coordinates from 1 image to the other. It is a linear mapping between the 2 images.



A H matrix has 8 degrees of freedom (even though it has 9 elements) because scaling is redundant, it does not change the mapping.

A Homography matrix is invertible, because if H maps image A to B, then H^{-1} maps B to A. (No information should be lost).

Let (x_1, y_1) & (x_2, y_2) be the corresponding points of the same feature in 2 images.

then -

$$\begin{bmatrix} x_2 \\ y_2 \\ 1 \end{bmatrix} = H \begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix}$$

$H \rightarrow$ homography matrix

$$\begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix}$$

It only works then the scene is planar (no significant depth variation) and the camera only rotates about its center (No translation in 3D).

In order to align images, we first need to find the homography matrix connecting the 2 images linearly.

- Feature Detection- A feature is a distinct information, which can be identified from an image. Such as corners, edges etc.
- Feature Mapping- After detecting them, comparing the descriptors of each feature point and finding the ones that match most closely. This can be done with ML algos like k-th nearest neighbor using cosine similarity.
- Now we can estimate the H matrix. 8 DOF and each points gives us x and y coordinates. Therefore we need at least 4 points for homography. If we have more than 4(as usually happens) we use least square optimization of the error to get the best fit. This is usually done with a DLT(direct linear transform) algorithm.

Algorithm

Step 1-Preprocessing

Load the 2 images using OpenCV.

We keep a copy of the original RGB for visualization.

Convert to inverted greyscale using BGR2GRAY and subtracting the value from 255. This is a standard practice when using OpenCV. As it

- Reduces complexity- as we only need to process the 1 channel instead of the 3 RGB channels. This reduces the data ORB has to process by 66%.
- Using Greyscale also highlights intensity gradients easily.

We have also inverted the greyscale to highlight dark features

Step 2-Feature detection.

As the bottom half of the image is of the same car, we will run into the problem of same feature(of car) getting highlighted instead of the surrounding road.

Therefore we mask the bottom 55% of the image and only focus on the top.

ORB (Oriented FAST and Rotated BRIEF): Initialize the ORB detector to find up to 1,000 keypoints.

Compute keypoints (distinct corners or edges) and their corresponding descriptors

Step 3- Feature mapping.

k-Nearest Neighbors For every feature in Image 1, find its two closest matches in Image 2.

A match is kept only if the distance to the closest neighbor is significantly smaller ($<75\%$) than the distance to the second closest neighbor. This removes ambiguous matches.

Step 4: Homography Estimation

Extract (x, y) coordinates of the filtered keypoints.

RANSAC (Random Sample Consensus):

- Select 4 random pairs of points to estimate a Homography matrix.
- Test how many other points fit this H within a 5-pixel threshold.

Step 5: Image Warping & Visualization

Use the H matrix to transform the pixels of Image 2.

Alignment: Warped Image 2 is mapped into the shape and perspective of Image 1.

Output: Display the matched keypoints (drawing lines between them) and the final side-by-side comparison of the reference and the warped result.

b) Distortion

We notice that the image is blurred in the direction of the motion. This is a likely result of motion blur. This can be caused due to High relative velocity wrt to shutter speed.

It degrades Homography estimation because it depends on features like sharp corners, stable keypoints and descriptors. Motion blur messes all these up.

- i) Feature detection goes wrong as ORB tries to detect corners using FAST detector, but corners lose sharp intensity gradients and shift.
- ii) Due to motion blur, descriptors from frame 1 change in frame 2, resulting in fewer inliers.
- iii) RANSAC gets warped and skewed.

We use blurring techniques like Gaussian blur, Median blur etc. often in OpenCV. Where we treat the image as a matrix and convolve a kernel through it, taking some average intensity values for the pixels inside the kernel, in order to blur the image. We can use the converse of this process in order to unblur a blurred image. We need to create an inverse of the kernel which when convolved with the blurred image gives the sharper image.

Let B be the blurred matrix (image) & A be the Original. Let K be the kernel, which is convolved w/ Original image.

$$B = K * A$$

Problems $\rightarrow$ convolution is usually not invertible as information is lost (specially in a blur)
 $\rightarrow$ Fortunately we have a restriction which we can place on blurring
i) It is in the direction of motion.

Usually blurring destroys information. Blurring on the second trail usually moderately damages details.

Algorithm-

Step 1-Preprocessing

Load the 2 images using OpenCV. Convert to inverted greyscale using BGR2GRAY and subtracting the value from 255.

Step 2- Estimate Unblur Kernel matrix.

For example, a linear kernel

$$K(x,y) = \begin{cases} 1/L & \text{(when in the direction of motion)} \\ 0 & \text{(otherwise)} \end{cases}$$

Step 3- Convolute the kernel over the input blurred image. To obtain The sharper image output.

Step 4- Filter the output image, using a normalization kernel or any Other filtering techniques.

Assuming a linear motion blur, when motion is performed in a straight Line. The blur kernel can be made with the motion blur angle θ , and blur length L . Blur direction - Camera motion direction,
blur length(L)= velocity $\times$ exposure time
Therefore using an IMU can give us direct measurements of acceleration and speed in camera frame.

Reference-

<https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=8405405>

No it isn't possible to find a depth map from homography as it assumes planar and rotational view. It collapses 3D images into a 2D projectional view, depth variation is not recorded per pixel.

Therefore to get a depth map we can work out an algorithm under the principle

“Points in the scene that are sharply in focus at a given image correspond to a specific depth”

Algorithm-

Step 1-Preprocess image

Load image with different focus settings and convert them to greyscale.

Step 2 - Iterate over every image of the stack. For each pixel location (x, y): Measure the **rate of intensity change** (sharpness) at that pixel. For each pixel determine, which image has the highest rate of change of intensity. This is the image which has that pixel with it in focus.

Step 3- The depth value of that pixel is mapped based on the corresponding focus position in the stack. So for pixel (x, y):

1. Identified image index = which image in stack had maximum sharpness.
2. That index is converted to a depth estimate

Higher focus distance = Greater depth

Step 4- Build a depth map using normalized values of where each pixel encodes depth.

Assumptions-

- Images are already well-aligned.
- Works best with textured scenes texture is needed to measure sharpness reliably

Reference - <https://github.com/tonyketcham/Depth-Mapper>

Q2)TRACKVILLE CONE HUNT

CODE-https://github.com/DiscoJK414/Raftaar_App/tree/main/Q2

a) Preprocessing

- i) Resize- To get all images in required dimensions, for easier processing.
- ii) Gaussian blur, from an appropriate sized kernel. This helps to reduce noise and smoothen out variation for better thresholding. As we are going to perform edge detection ahead.
- iii) BGR to HSV, This separates the colors from their brightness. This makes images stable under lighting differences.

b) Our objective is to detect the colored cones. Blue Yellow and Orange have different HSV values, if they were processed under 1 single mask, thresholding each color would become very difficult. Blue cones show left boundary. Yellow cones show right boundary and Orange cones shows start/end markers

If we combined them into one mask,

- The threshold range would become too wide
- False positive results possible.

Hence we created 3 different masks which would give color filtered images for better processing of the cones.

c) There are many types of edge detection algorithms and techniques, a few of them are-

i) **Sobels operator**

Sobel's filter is a approximation of the concept of gradient with smoothing. The **3x3 kernel detects gradients** in XY direction. The kernels are designed to maximally respond to edges running in those particular directions relative to the pixel grid.

They are then combined together to get the absolute magnitude of the gradient at each point. Given by

10:46 PM Mon 2 Mar 29%

G_x (Horizontal) G_y (Vertical)

-1	0	1
-2	0	2
-1	0	1

rotated

-1	-2	-1
0	0	0
1	2	1

$|G| = \sqrt{G_x^2 + G_y^2}$

↳ approximated

$|G| = |G_x| + |G_y|$ (faster to compute)

↓ angle of edge given by

$\theta = \tan^{-1}\left(\frac{G_y}{G_x}\right)$

input image

output image

kernel: bottom sobel

58 + 53 + 67
x -1 x -2 x -1

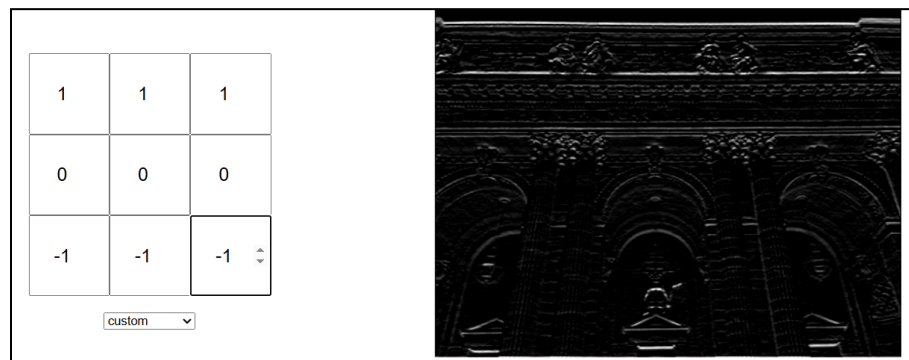
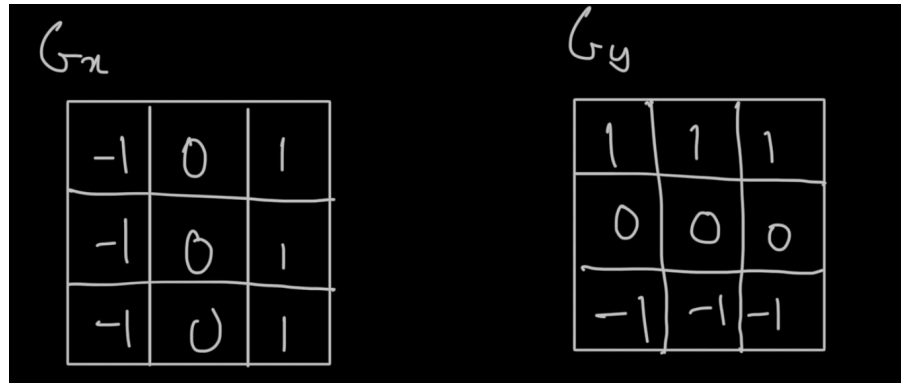
+ 158 + 137 + 92
x 0 x 0 x 0

+ 177 + 177 + 172
x 1 x 2 x 1

= 472

ii) Prewitt's Operator

Same idea as sobel's filter. More suitable for noiseless and well-contrasted images. Kernels are different



iii) Robert Cross Operator

It performs a 2x2 convolution over the image, which makes it quicker and easier to compute.

It highlights regions of the **fine-grained details, sharp edges, and rapid changes in intensity, which usually correspond to edges.**

The input is usually a **greyscale image**. Similar to sobel and Prewitt's operator it has 2 kernels corresponding to XY.

These kernels are designed to maximally respond to edges running at 45° to the pixel grid.

Handwritten mathematical formulas for Sobel operators:

$$G_x = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \quad G_y = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$$

$$|G| = \sqrt{G_x^2 + G_y^2}$$

$$\theta = \tan^{-1}\left(\frac{G_y}{G_x}\right) - \frac{3\pi}{4}$$

iv) **Laplacian of Gaussian**

LOG operator finds the optimal filter of edge detection by the signal to noise ratio of the image.

First Gaussian blur.

Then high pass filter to Laplacian operator (Second Derivative Method)

v) **Canny Operator**

The basic idea uses a Gaussian function to smooth image firstly.

Then the **maximum value of first derivative also**

corresponds to the minimum of the first derivative, i.e. both points with dramatic change of gray-scale (strong edge) and points with slight change of grayscale (weak edges) correspond to the second derivative zero-crossing point. Thus these two thresholds are used to detect strong edges and weak edges. The fact that Canny algorithm is not susceptible to noise interference enables its ability to detect true weak edges.

References-

<https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=7210349>

According to the conclusions of this paper-

Purely gradient based edge detection algorithms like Robert's algorithm have a major drawback of being very sensitive to noise and produces weak responses for genuine edges, Sobel's algorithm: less sensitive to noise but computationally it is slower, Prewitt's algorithm: the response of the prewitt kernel is too small also sensitive.

Canny edge detection approach performs better than all operators under all scenarios.

This can be credited to the fact that it-

a) Blurs first then finds gradients(Noise resistant)

b) Non maximum suppression(1 pixel wide clean edges)

d) Now we combine all these steps in order to detect the edges and for depth estimation we work under the core idea of

“Absolute depth is ambiguous from a single image, Relative depth(depth ratios between pixels) is scale invariant and easier to compute”

Reference -

<https://www.sciencedirect.com/science/article/pii/S1047320322000190?via%3Dihub>

1:32 AM Tue 3 Mar 23% 🔍 Search

Suppose absolute depth map is D , (for cones)

$$D = [d_1, d_2, d_3 \dots d_N]$$

$\therefore$ Scaled depth map $D \propto cD$

$$\Rightarrow cD = [cd_1, cd_2, cd_3 \dots cd_N]$$

Both produce same 2D image.

Normalize depth with geometric mean as it is a ratio

$$g(D) = \left(\prod_{i=1}^N d_i \right)^{\frac{1}{N}}$$

1:32 AM Tue 3 Mar 23% 🔍 Search

$\therefore$ Normalized depth = $R = \frac{D_i}{g(D)}$ $\rightarrow$ even if scaled R is same

$\therefore$ Depth map = $[R_1, R_2 \dots R_N]$ $\propto \frac{D}{g(D)}$

$\Rightarrow$ pin hole camera

$$h_i = \frac{fH}{d_i}$$

$$d_i \propto \frac{1}{h_i}$$

$h \rightarrow$ bounding box

$H \rightarrow$ real cone height

$f \rightarrow$ focal length

Let $r_{ij} = \frac{d_i}{d_j}$ (relative depth b/w 2 cones)

We can see that we can use bounding boxes instead of true heights for depth estimation. This solves quite the bit of problems.

$$k_{ij} = \frac{h_j}{h_i} \\ = h_j \times \frac{1}{h_i} = 0 \cdot \left(\frac{1}{h_i}\right)^T \rightarrow \underline{\text{rank 1}}$$

This is peak, Now we can use the ideas of **relative depth from the paper and apply it at object level instead of pixel level.**

The proposed system draws conceptual inspiration from the relative depth formulation introduced by Lee & Kim (2022),

Algorithm-

- 1)Preprocessing- We first read the image using opencv, resize it, gaussian blur it and convert it to HSV(as discussed before).
- 2)Then we define 3 masks corresponding to each color of the cones. Using upper and lower bounds of HSV. This basically gives us a way to color filter the HSV image. The ranges for orange and yellow are very close, therefore we make their masks mutually exclusive.
- 3)Now we bitwise AND the HSV image with itself, using each of the 3 masks applied. This is the color filtered image.
- 4)Next we combine all the masks using bitwise OR, so we now have a cone filter(mask that passes cones regardless of color)
We then use Canny Edge detection on this image. Next we find external contours of the cones.
- 5)Create empty list height to store bounding box heights.

Create empty list box to store bounding box data.

Iterate over each contour-

i) Compute area and if area is < 30 discard it.(its prolly noise or a blob)

ii) Using the bounding data calculate aspect ratio(height/width)

If $AR < 1$ OR $AR > 8$ discard contour(its prolly not a cone as they are triangular).

Now we store the data in the box data.

6) Now we use the depth estimation we developed. We annotate the center of each cone and write coords and depth.

7) We display the 3 masked images and the annotated image as Output.

Q3) TRACKING THE TRACK AHH

CODE-https://github.com/DiscoJK414/Raftaar_App/tree/main/Q3

a) With its accelerometer(linear acceleration) Gyroscope(angular rate of change) and magnetometer(local MF wrt to earth's MF), the IMU(Inertial Measurement Unit) which is mounted on the chassis provides **Body fixed Coordinate System**. This type of reference frame consists of an origin that is typically placed at the platform's centre of gravity, and three orthogonal axes.

Transformation from body frame to world coordinates is basically rotation

$$R = \begin{bmatrix} \cos \Psi & -\sin \Psi \\ \sin \Psi & \cos \Psi \end{bmatrix}$$

where Ψ is the yaw angle measured by $\Psi = \tan^{-1}\left(\frac{m_y}{m_x}\right)$ by magnetometer readings.

Table 2.5: *Body fixed ENU Reference Frame.*

Origin	Fixed at the vehicle's center of gravity.
X-axis	Points in the Longitudinal direction.
Y-axis	Points in the Lateral direction.
Z-axis	Points Up.

GPS(Global Positioning System) offers absolute positioning, velocity and time to navigate a car. World Geodetic System(WGS84) is the standard geodetic coordinate system for GPS.The **Geodetic coordinate system** is a set of polar coordinates, that accounts for the first-order effect of Earth being an ellipsoid rather than a sphere. ECEF-g is more convenient to express positions in Earth's immediate surroundings, so it has been chosen as standard reference for satellites positioning systems. The ECEF(Earth Center earth first)-g position of an object is described in terms latitude (ϕ), longitude (λ), and altitude or elevation (h).
anThese are fundamentally different.

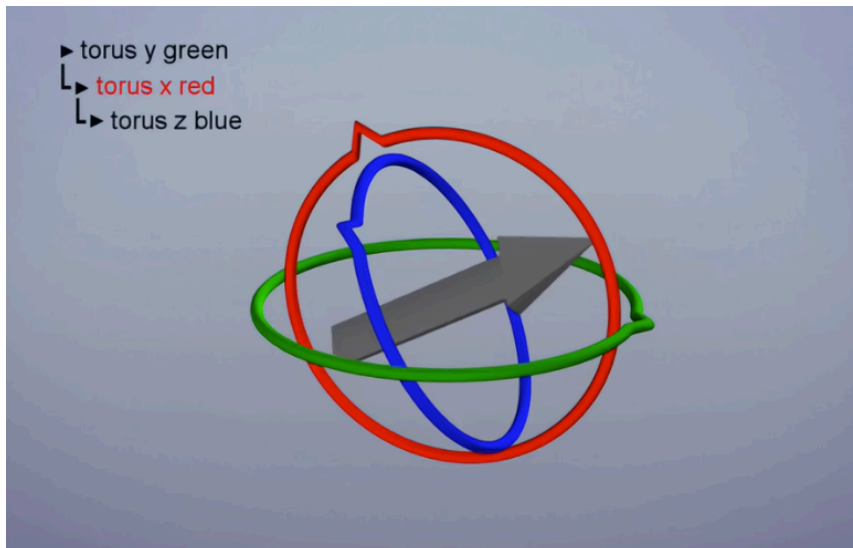
1) Frame Misalignment-Acceleration measured by the IMU describes motion **relative to the vehicle's orientation**, not relative to the Earth's

If the car accelerates **forward**, the IMU records acceleration along the **body x-axis**. But the direction of this axis depends on the vehicle's **heading**.

If the car is facing **Southwest**, the same forward acceleration is a **combination of South and West** in the global frame.

Therefore, if we tried to integrate the IMU acceleration directly to update position without transforming it, the system would **update the vehicle's position in the wrong direction** whenever the car changes orientation.

Euler angles is basically when we describe rotations as sequence of rotations about the three axes.

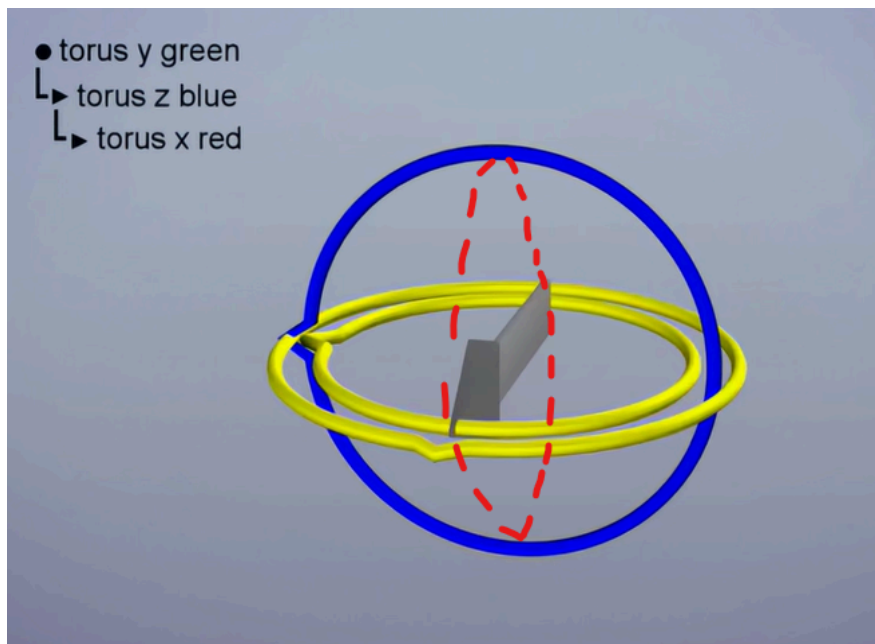
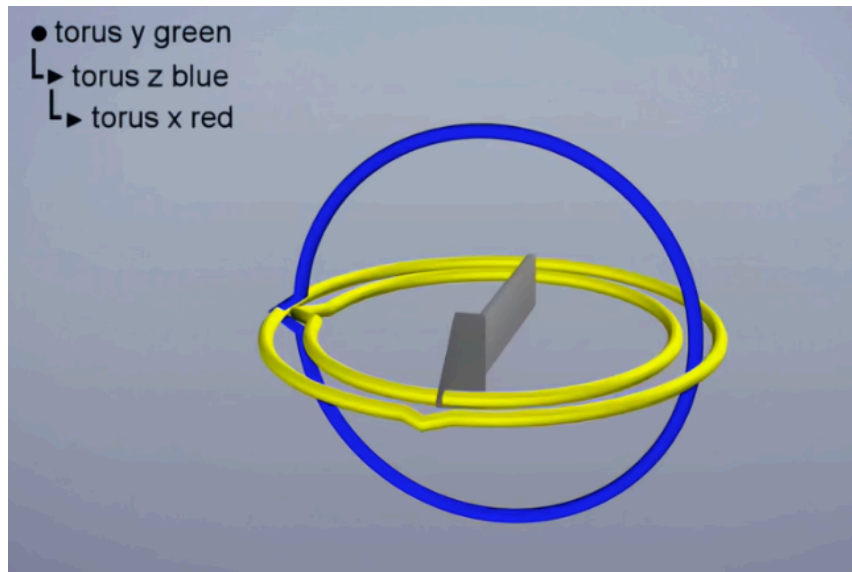


In a vehicle navigation system, the orientation of the car is often estimated using an **IMU**. If the orientation is represented using Euler angles, the sequence might be:

$$R = R_z(\text{yaw}) R_y(\text{pitch}) R_x(\text{roll})$$

Now the biggest problem with euler angles is, **Gimbal Locks**, this is when 2 of the axes line up and the system essentially loses 1 degree of freedom.

Suppose, pitch angle approaches $\pm 90^\circ$, so pitch and roll axis line up. Now independent rotations about roll axis is literally incomprehensible to the computer because it has lost 1 degree of freedom.



(Dotted line is missing DOF)

During **high speed turning where roll and yaw(especially in a hairpin turn) rapidly increases or fast braking where pitch suddenly increases**, it is possible in the system for the axis to line up

and cause gimbal locks.

Quaternions are basically 4 dimensional complex numbers, analogous to how $a+bi$ are 2D complex numbers. Similarly to how $e^{i\theta}$ simplifies 2D rotation, quaternions simplify 3D rotations.. The direction of the rotation axis and the size of the angle are encoded in a single vector, where the direction of this vector determines the rotation axis, and its size determines the angle.

$$q = \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} = \begin{bmatrix} \cos(\mu/2) \\ (\mu_x/\mu) \sin(\mu/2) \\ (\mu_y/\mu) \sin(\mu/2) \\ (\mu_z/\mu) \sin(\mu/2) \end{bmatrix},$$

References-

<https://webthesis.biblio.polito.it/17821/1/tesi.pdf>

<https://www.sciencedirect.com/science/article/pii/S1474667015350540>

<https://www.youtube.com/watch?v=zjMuIxRvygQ>

<https://www.youtube.com/watch?v=zc8b2Jo7mno>

b)

We want 2D position and velocity
 $\therefore$ State vector s
$$s = [x, y, v_x, v_y, \psi]^T$$

States Primarily Informed by the IMU

The IMU is the primary driver of the vehicle's short-term dynamic states. Because it measures forces and rates directly in the body frame, it is suited to capturing high-frequency components of velocity and position states that may occur between GPS updates. This is done by integrating the acceleration values to update velocity and position vectors.

The Yaw is calculated by the magnetometer in the IMU, By integrating the angular rate measurement, the system can track changes in orientation with extremely high precision. The gyroscope ensures that the system knows which way it is facing at all times.

The IMU's ability to provide these updates at hundreds of hertz is what enables smooth navigation and stable feedback control for autonomous driving.

States Primarily Informed by GPS

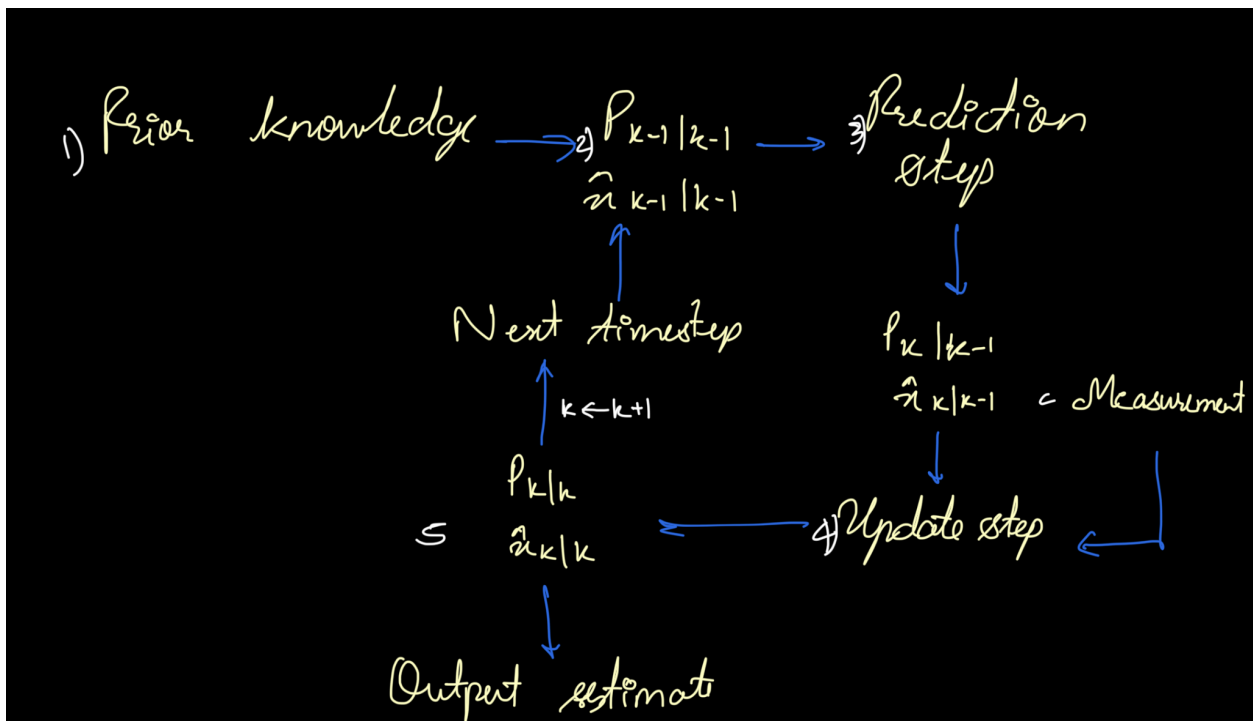
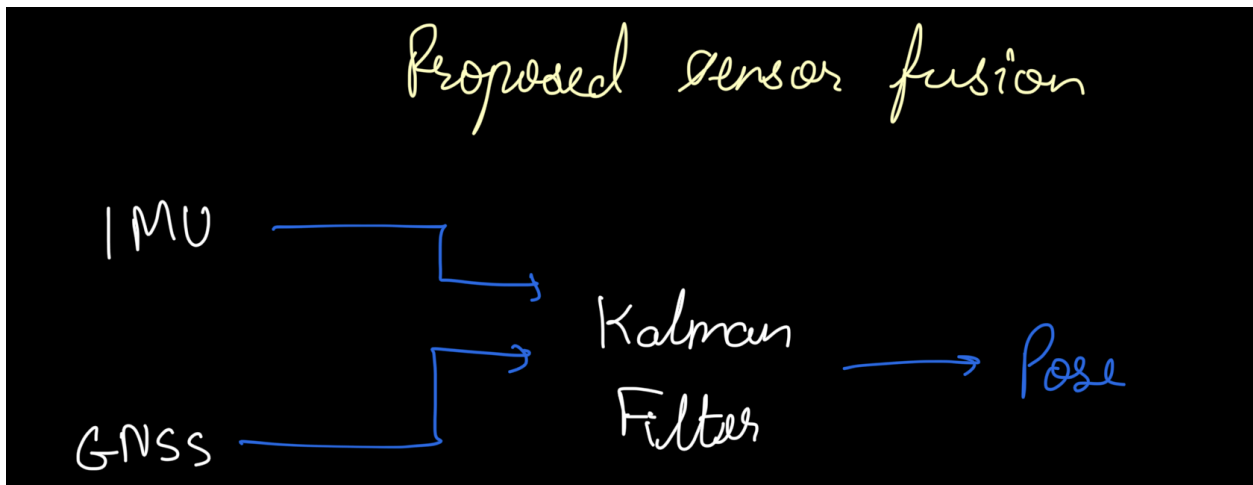
The position states measurement(P_x and P_y) are made by GPS. Every time a GPS signal is received, it provides a measurement of the vehicle's location with an absolute accuracy typically in the range of 1 to 5 meters for standard receivers. This measurement is used to correct the errors that have accumulated in the IMU-based prediction. Sensor biases that have been accumulated by IMUs(as we get values by integration, the noise also gets amplified) can also be corrected.

When GPS signal is lost

The system transitions into a state of pure "dead reckoning". where only IMU data is used to propagate the state forward, using the velocity and position estimates.

Velocity error grows linearly over time. The position error, being the integral of the velocity, grows quadratically over time. Drift is often the most significant contributor to position error during a short outage. This causes the system to incorrectly project the vehicle's forward motion, leading to a "curved" path in the estimated coordinates even if the vehicle is driving in a straight line.

c)



as seen before

$$R(\Psi) = \begin{bmatrix} \cos \Psi & -\sin \Psi \\ \sin \Psi & \cos \Psi \end{bmatrix}$$

body frame $\rightarrow$ global frame

$v_{\text{body}} = \begin{bmatrix} v \\ 0 \end{bmatrix}$

car always moves forward in its frame.

$\therefore v_{\text{global}} = R(\Psi) v_{\text{body}}$

$$v_G = \begin{bmatrix} \cos \Psi & -\sin \Psi \\ \sin \Psi & \cos \Psi \end{bmatrix} \begin{bmatrix} v \\ 0 \end{bmatrix}$$

$\therefore v_G = \begin{bmatrix} v \cos \Psi \\ v \sin \Psi \end{bmatrix}$

$\therefore$ Similarly position

$$\begin{bmatrix} p_x \\ p_y \end{bmatrix} = \begin{bmatrix} p_{x-1} \\ p_{y-1} \end{bmatrix} + R(\Psi) \begin{bmatrix} v \\ 0 \end{bmatrix} \Delta t$$

As most things in life, we can't use a linear interpolation here because the rotation matrix depends on the $\cos(\text{Yaw angle})$ which induces non-linearity. So we use Unscented Kalman Filter (UKF). Instead of

linearizing like the Kalman Filter, the **UKF uses “sigma points”** to track how uncertainty moves through a nonlinear function. This when propagated through the true non-linear system, the filter is capable to capture the posterior mean and covariance accurately to the 3rd order Taylor series expansion for any non-linearity

Algorithm-

1) Prediction step

Sigma points are given by-

$$\bar{X}_{t-1} = [x_{t-1} \quad x_{t-1} \pm \sqrt{(n + \kappa)P_{t-1}}]$$

So we create sigma points about the mean, according to covariance P. This basically creates an uncertainty cloud. For n dimensions of state vector we create 2n+1 sigma points. In our case, state vector has 5 dimensions so we create 11 points.

Now these sigma points are pushed into the function model, this gives us predicted sigma points.

$$\bar{x}_{t|t-1} = f(x_{t-1}, u_{t-1})$$

We use a weighted average of these predicted sigma points to get the predicted state.

$$x_t^- = \sum_{i=0}^{2n} W_i^m \bar{x}_{i,t|t-1}$$

$$P_t^- = \sum_{i=0}^{2n} W_i^c (\bar{x}_{i,t|t-1} - x_t^-) (\bar{x}_{i,t|t-1} - x_t^-)^T + Q,$$

where x_t^- represents the predicted mean, P_t^- represents the predicted covariance, and W_i^m and W_i^c are the weights for the mean and covariance, respectively, associated with the i-th sigma point.

P-t gives use how far each sigma point is from the mean

This gives the **new uncertainty of the predicted state**. Then we add **process noise Q** to account for model imperfections.

$$x_k = g(u_k, x_{k-1}) + \Sigma_k$$

$x_k \rightarrow$ current state

$u_{k-1} \rightarrow$ control input

$x_{k-1} \rightarrow$ previous state

$\Sigma_k \rightarrow$ process noise.

$$z = H(x_k) + \delta_k$$

$$p(\varepsilon_k) \sim N(0, R_k)$$

$$p(\delta_k) \sim N(0, Q_k)$$

2) Measurement stage-

Now we generate sigma points about the predicted mean.

$$\bar{X}_t = [\bar{x}_t \quad \bar{x}_t \pm \sqrt{n + \kappa} P_t]$$

where $\bar{x}_t$ represents the predicted mean from the time update at time t and P_t denotes the predicted covariance. Instead of representing uncertainty with just a covariance matrix, we create **$2n + 1$ sample points** around the mean.

These points represent the **uncertainty distribution of the state**.

Pass Sigma Points Through Measurement Model

The function $g(x)$ is the measurement model. It converts the state into what the sensor would measure.

$$\bar{Y}_t = g(\bar{X}_t)$$

So each sigma point is converted into a predicted measurement. Now we do the same weighted average. To get average predicted measurement.

$$\bar{y}_t = \sum_{i=0}^{2n} W_i^m \bar{Y}_{i,t}$$

Compute Measurement Covariance

$$P_{y_t} = \sum_{i=0}^{2n} W_i^c (\bar{Y}_{i,t} - \bar{y}_t) \cdot (\bar{Y}_{i,t} - \bar{y}_t)^T + R$$

This computes the **uncertainty in the predicted measurement**. It measures how spread out the predicted measurements are. Then we add **measurement noise R**

$$P_{x_t y_t} = \sum_{i=0}^{2n} W_i^c (\bar{X}_{i,t} - \bar{x}_t) \cdot (\bar{Y}_{i,t} - \bar{y}_t)^T$$

$P_{x_t y_t}$. This matrix measures **how uncertainty in the state affects uncertainty in the measurements**.

Each sigma point contributes

$$(\bar{X}_{i,t} - \bar{x}_t)$$

This measures how far the state sigma point is from the state mean.

$$(\bar{Y}_{i,t} - \bar{y}_t)$$

This measures how far the predicted measurement is from the measurement mean.

Multiplying them together gives, when state moves in some direction how does the prediction move.

3) This helps us compute Kalman Gain

$$K_t = P_{x_t y_t} \cdot (P_{y_t})^{-1}$$

The **Kalman Gain** determines how much we trust the prediction vs the measurement

If measurement noise is small, trust sensors more.

If measurement noise is large, trust prediction more.

Then we compute error, between real measurement and predicted measurement, now we can update the prediction and Covariance by adding the error.

Process Noise Q represents uncertainty in the system model itself-

Unmodeled dynamics -friction, temperature effects on thrust, wind gusts, road bumps, sudden accelerations etc.

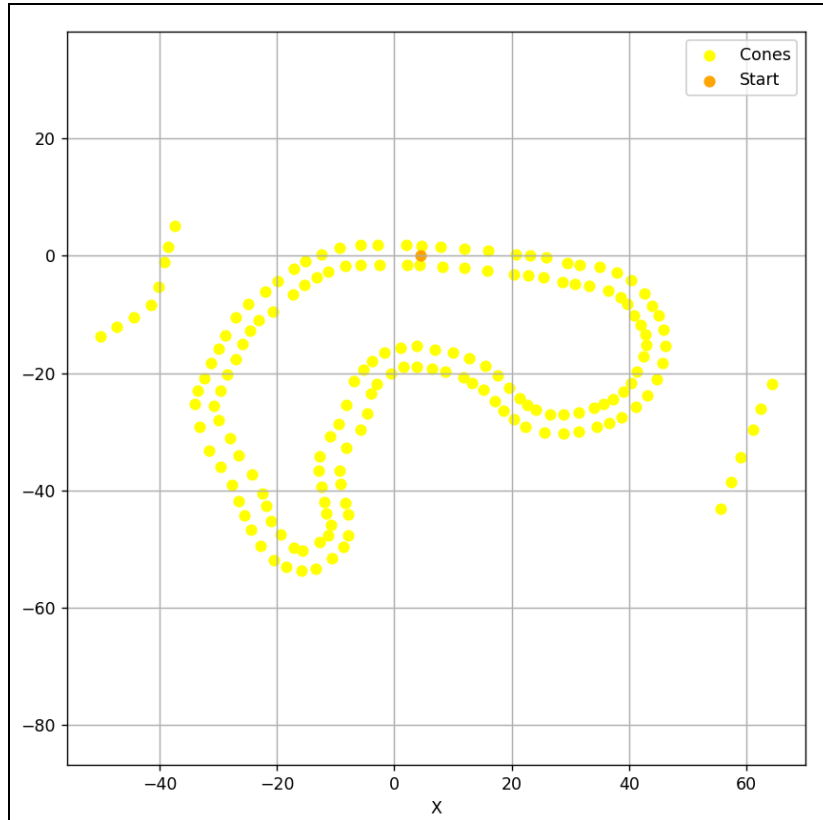
Actuator imperfections -a motor commanded to 5 rpm doesn't spin at exactly 5 rpm

Scenario	Effect
Q too small	Filter overtrusts the model low to respond to real changes
Q too large	Filter distrusts the model noisy, jittery estimates
R too small	Filter overtrusts sensors, estimate chases noisy measurements, unstable
R too large	Filter distrusts sensors ignores real data, estimate drifts with model errors

References - <https://webthesis.biblio.polito.it/17821/1/tesi.pdf>
<https://arxiv.org/pdf/2405.08119>

Q4)PLAN THE PATH

a)The system has given us a set of coordinates and specified them as cones or starting points. Plotting them we get



Observations-

The distance between cones on the **same boundary** is typically smaller than the distance between cones across the track.

The track boundaries are **approximately parallel** locally.

The safe path lies approximately **midway between the two boundaries**.

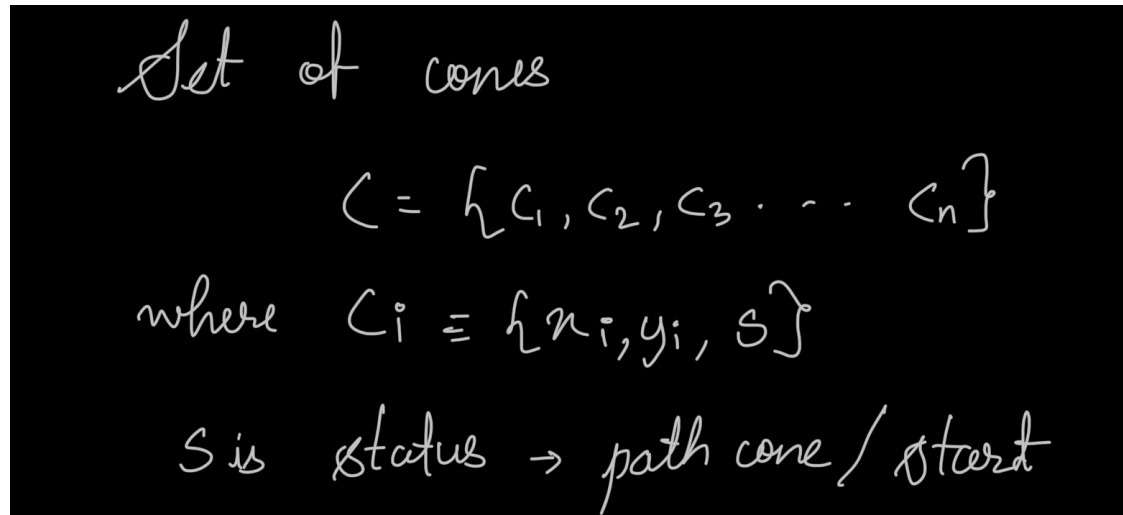
The path of the track forms a **continuous smooth curve**, even when the track turns sharply.

So now we can clearly see the track boundaries, we need the autonomous system to differentiate between the left and right boundary and how to construct a safe region between them. For this i propose integrating 2 ideas- 1) Voronoi's diagram for global structure
2) Local Tangent cosine similarity.

These 2 implemented together should in theory be a robust algo.

Algorithm

Given data -



Handwritten text on a black background:

Set of cones

$$C = \{c_1, c_2, c_3, \dots, c_n\}$$

where $c_i = \{x_i, y_i, s\}$

s is status $\rightarrow$ path cone / start

1)Preprocessing -

Step 1-The algorithm evaluates the **local neighborhood structure around each cone**. To measure this, the algorithm builds a KDTree. For each cone, the algorithm counts how many other cones lie within a predefined distance threshold. If a cone has fewer neighbors than the minimum required number, it is considered an outlier and removed.

2)-Construction of Voronoi's Diagram-A Voronoi diagram divides the plane into regions where every point is closest to a particular cone. A Voronoi edge consists of points that are **equidistant from two seed points**.

This property is useful in our analysis.

The midpoint between two cones whose separation corresponds to the track width lies approximately near the center of the drivable corridor. The set of all such midpoints therefore forms an approximately the safest region for the vehicle to drive.

1) Compute distances to nearest neighbour.

$$d_i = \min |c_i - c_j|$$

$$\text{if } d_{\min} < d < d_{\max}$$

↳ probably opposite track boundaries.

$d_{\min} \rightarrow$ minimum possible cone spacing

$d_{\max} \rightarrow$ track width

If the cones are really close together, they likely belong to the **same boundary**. If they are extremely far apart, they are probably unrelated.

For cones where $d_{\min} < d < d_{\max}$, we calculate the midpoint for the center line. These are stored for processing

3) Constructing the center line-

We have an unordered set of midpoints from the previous step, now we connect them using nearest-neighbor traversal.

Midpoint

$$m = \frac{c_i + c_j}{2}$$

$\perp^{\text{th}}$ direction

$$v = c_j - c_i$$

$$v_{\perp} = (-v_y, v_x) \rightarrow \text{direction of Voronoi edge}$$

Start from the midpoint closest to the start cone, a set of nearby candidate midpoints is identified using the KDTree. These candidates are evaluated based on two criteria:

1. The candidate should not have been used already.
2. The candidate should not require an excessively large step distance.

This creates a **continuous curve through the track center**.

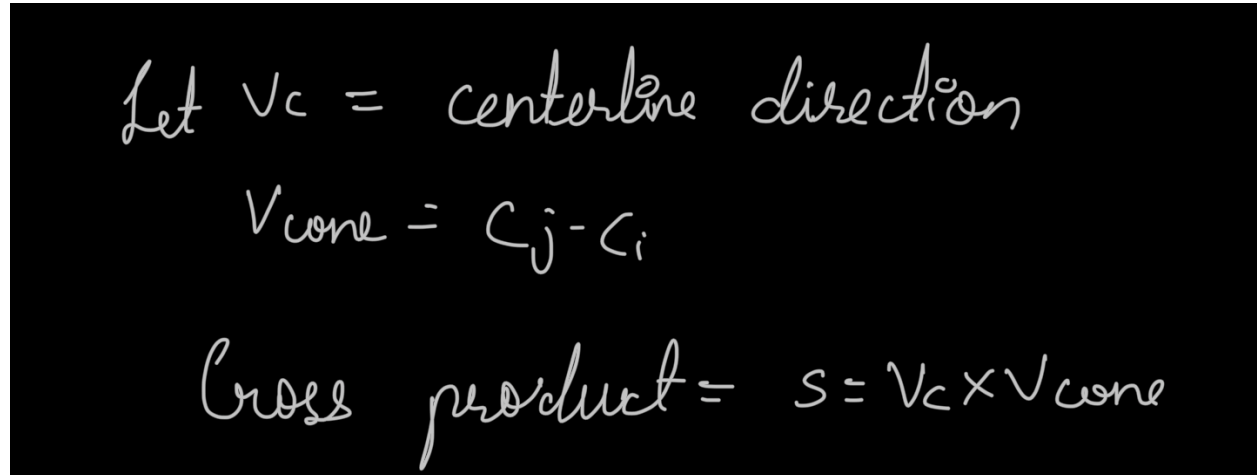
If the path already has a heading, evaluates how well each candidate continues that direction. This is done using the **dot product between vectors**.

We assign a score that balances forward alignment and short step distance. The candidate with the best score is selected as the next point on the path.

4) For each cone, the algorithm examines its position relative to the local centerline direction. We need to determine which cone is on the **left boundary** and which is on the **right boundary**.

Use orientation relative to centerline

At each centerline point, voronoi edge direction should be vehicle direction



Let $V_c = \text{centerline direction}$
 $V_{\text{cone}} = C_j - C_i$
Cross product = $S = V_c \times V_{\text{cone}}$

If the cross product is positive, the cone lies on one side of the direction of travel. If it is negative, the cone lies on the opposite side.

By applying this test to all cones, we divide the set of cones into two groups:

- cones belonging to the **left boundary**
- cones belonging to the **right boundary**

Now, these unordered but classified points must be connected into continuous boundaries. This can be done with tangent continuity, The direction of travel changes gradually rather than abruptly. Therefore, the next cone on the boundary should lie in roughly the **same direction as the current boundary tangent**.

To measure this alignment, we compute the **cosine similarity between vectors**.

Current direction $v_1 = p_i - p_{i-1}$

Candidate direction $v_2 = p_j - p_i$

$\cos \theta = \frac{v_1 \cdot v_2}{|v_1| |v_2|}$

$\begin{cases} \sim 1 & \text{similar direction} \\ \sim 0 & \text{sharp turn} \\ < 0 & \text{wrong} \end{cases}$

These are then ordered according to the cosine similarity to form the left and right edges.

5) Once both boundaries are known, the drivable region of the track is simply the area between them. This produces a final centerline that lies exactly halfway between the two boundaries.

6) Boundary cones are unordered and must be arranged sequentially.

Build a KDTree of boundary points.

Choose a starting cone with smallest nearest neighbor distance.

Iteratively append the nearest unused neighbor within maxhop.

7) Finally, the algorithm visualizes the reconstructed track using a plot.

b) One assumption applied in racing line optimization is that the optimal path will be a combination of the maximum curvature path (MCP) and the shortest distance path (SP). So we can find an optimal path with the weighted tradeoff between MCP and SP. Which has the following features-

1. Allow a high degree of geometric flexibility in racing line representation
2. Produce a racing line that appears realistic and smooth (C^2 continuity)
3. Make full utilization of the available track surface, meeting all apexes and borders where necessary
4. Require minimal setup procedure allowing for the analysis of a large number of tracks and vehicles, with a fully automated solution process.

This racing line optimization draws inspiration from this paper -

<https://dl.acm.org/doi/pdf/10.1145/2815474.2815476>

Algorithm-

- 1) The centerline and the track boundaries are sampled, to create evenly spaced points along the track. The original reconstruction might be inconsistent.

- a) First we find the distances between consecutive points, Then calculate the cumulative arc length and normalize it to be (0,1)

$$d_i = |p_{i+1} - p_i|$$

$$s_i = \sum_{k=1}^i d_k$$

$\xrightarrow{\text{Norm}}$

$$s_i = \frac{s_i}{s_{\max}}$$

- b) Then we linearly interpolate the x and y coordinates. This gives us N evenly spaced points on the track.
- 2) To allow the racing line to move left or right inside the track, we compute **normal vectors** along the centerline.
- a) First we calculate the tangents using
- $$t_i = p_{i+1} - p_i$$
- And normalize the values.
- b) Then we compute the normals using a simple rotation matrix

$$\begin{bmatrix} -t_{iy} \\ t_{ix} \end{bmatrix}$$

- c) Now we determine the width of the track, this allows us to find how far laterally we can move from the centerline

$$d_l = |center_i - left_i|$$

$$d_r = |center_i - right_i|$$

$$w = \min(d_l, d_r)$$

This allows us to parameterize the path we take, with the centerline as the anchor

Each point on the track is given by

$$P_i = C_i + \alpha_i n_i$$

C_i = Center line point
 α_i = lateral offset
 n_i = normal vector

This is peak now we can just optimize alpha for each point, instead of optimizing for both x and y coords.

- d) The goal of a racing line is to **minimize curvature**, which leads to smoother paths and higher allowable speeds.

$$a_i = P_i - P_{i-1}$$

$$b_i = P_{i+1} - P_i$$

curvature

$$k_i = \frac{a_i \times b_i}{|a_i|^3}$$

For minimizing we define a cost function

$$J(\alpha) = \sum k^2$$

Now this is just a problem of bounded optimization, we have to optimize $J(\alpha)$, given the constraints-

$$\alpha_{min} \leq \alpha_i \leq \alpha_{max}$$

The optimization begins with an initial guess $\alpha=0$, which corresponds to the centerline. It adjusts α till curvature cost converges.

e) Now we plot the output, which is the optimized racing line.